

Name #1: _____

Name #2: _____

In-class Quiz #05: RAM & ROM

Directives

1. Write the **directive** to create a **32-bit constant** named **Const1** initialized to **0x1234ABCD**.

Const1 DCD 0x1234ABCD

2. Write the **directive** to create a **16-bit constant** named **Const2** initialized to **0x5678**.

Const2 DCW 0x5678

3. Write the **directive** to create a **32-bit variable** named **Var3**.

Var3 SPACE 4

4. Write the **directive** to create a **16-bit variable** named **Var4**.

Var4 SPACE 2

5. Please fill in the values in the according memory after the following directives. Assume the address of Const5 is 0x00000284. If the memory is not defined or initialized, fill in with "00". (Using hexadecimals for all the values, don't need to write the prefix of "0x"; one cell means a byte)

Const5 DCD 0x56781234
Const6 DCW 0xABCD, 0x1357

Address	Contents							
0x00000284	34	12	78	56	CD	AB	57	13
0x0000028C	00	00	00	00	00	00	00	00

Load/Store Registers from/into RAM/ROM

6. Write the assembly code to **load** register **R0** with the value of the **32-bit** constant named **const1**.

```
LDR R1, =Const1  
LDR R0, [R1]
```

7. Write the assembly code to **store** the value in register **R0** to the **32-bit** variable named **var3**.

```
LDR R1, =var3  
LDR R0, [R1]
```

8. Assume you have used the following directives to define a vector of constants.

```
Const7      DCD    0x12, 0x34, 0x56, 0x78
```

Please complete the following instructions to read "0x56" into register R0

```
LDR R1, =Const7  
LDR R0, [R1, #8]
```

(Hint: The Cortex-M processor supports offset addressing. An *offset* from the *base address* can be used to generate a relative address. The instruction

```
LDR{type}{cond} Rt, [Rn{, #offset}]
```

will fetch the contents at the address $[Rn] + \text{#offset}$ into Rt, here $[Rn]$ means the address saved in Rn)

Programming

9. Write the assembly program to accomplish the following C instruction: (Both const2 and var4 are 16 bits and they already defined and initialized.)

```
var4 = const2;
```

10. Write the assembly program to create an empty array, which can contain 10 32-bit elements, and assign the number 1 to 10 to the corresponding elements. You can refer to the following C code:

```
int numbers[10];  
// Assigning numbers 1 to 10 into the array  
for (int i = 0; i < 10; i++) {  
    numbers[i] = i + 1;  
}
```